Apply filters to SQL queries

Project description

In this project, I took on the role of a security professional at a large organisation investigating potential security issues involving employee devices and login attempts.

My task was to use SQL queries to examine information stored in the `employees` and `log_in_attempts` tables. By filtering database records, I aimed to identify relevant login activity and employee information that could assist in investigating potential security issues.

Retrieve after hours failed login attempts

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE login_time > '18:00' AND success = 0;
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |       0 |
|       18 | pwashing | 2022-05-11 | 19:28:50   | US      | 192.168.66.142  |       0 |
|       20 | tshah    | 2022-05-12 | 18:56:36   | MEXICO  | 192.168.109.50  |       0 |
|       28 | aestrada | 2022-05-09 | 19:28:12   | MEXICO  | 192.168.27.57   |       0 |
|       34 | drosas   | 2022-05-11 | 21:02:04   | US      | 192.168.45.93   |       0 |
|       42 | cgriffin | 2022-05-09 | 23:04:05   | US      | 192.168.4.157   |       0 |
|       52 | cjackson | 2022-05-10 | 22:07:07   | CAN     | 192.168.58.57   |       0 |
|       69 | wjaffrey | 2022-05-11 | 19:55:15   | USA     | 192.168.100.17  |       0 |
|       82 | abernard | 2022-05-12 | 23:38:46   | MEX     | 192.168.234.49  |       0 |
|       87 | apatel   | 2022-05-08 | 22:38:31   | CANADA  | 192.168.132.153 |       0 |
|       96 | ivelasco | 2022-05-09 | 22:36:36   | CAN     | 192.168.84.194  |       0 |
|      104 | asundara | 2022-05-11 | 18:38:07   | US      | 192.168.96.200  |       0 |
```

I used `SELECT *` to retrieve all columns from the `log_in_attempts` table using the `FROM` statement.

I then used the `WHERE` clause to filter login attempts made after 18:00 (6 PM). Finally, I used the `AND` operator to add another condition, `success = 0`, which filters the results to show only failed login attempts. In this table, 0 represents a failed login attempt, while 1 represents a successful login attempt.

This query allows me to identify failed login attempts that occurred after 6 PM.

Retrieve login attempts on specific dates

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

I used `SELECT *` to retrieve all columns from the `log_in_attempts` table, which I specified using the `FROM` statement.

I then used the `WHERE` clause to filter login attempts made on `2022-05-09`. I also used the `OR` operator to include login attempts made on `2022-05-08`.

The `OR` operator allows the query to return records that match either of the two specified dates.

This query allows me to identify login attempts made on two separate days.

Retrieve login attempts outside of Mexico

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE NOT country LIKE 'MEX%';
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |       1 |
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |       0 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |       1 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |       0 |
|        5 | jrafael  | 2022-05-11 | 03:05:59   | CANADA  | 192.168.86.232  |       0 |
```

I used `SELECT *` to retrieve all columns from the `log_in_attempts` table, which I specified using the `FROM` statement.

I then used the `WHERE` clause with `NOT LIKE 'MEX%'` to filter out login attempts originating from Mexico. The `NOT` operator excludes matching records, while `LIKE` allows me to search for a specific text pattern. The `%` wildcard ensures that any country value beginning with `MEX` is excluded, regardless of what characters follow.

This query allows me to identify login attempts originating from countries outside of Mexico.

Retrieve employees in Marketing

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Marketing' AND office LIKE 'East%';
+-------------+--------------+----------+------------+----------+
| employee_id | device_id    | username | department | office   |
+-------------+--------------+----------+------------+----------+
|        1000 | a320b137c219 | elarson  | Marketing  | East-170 |
|        1052 | a192b174c940 | jdarosa  | Marketing  | East-195 |
|        1075 | x573y883z772 | fbautist | Marketing  | East-267 |
|        1088 | k8651965m233 | rgosh    | Marketing  | East-157 |
|        1103 | NULL         | randerss | Marketing  | East-460 |
|        1156 | a184b775c707 | dellery  | Marketing  | East-417 |
|        1163 | h679i515j339 | cwilliam | Marketing  | East-216 |
+-------------+--------------+----------+------------+----------+
7 rows in set (0.001 sec)
```

I used `SELECT *` to retrieve all columns from the `employees` table, which I specified using the `FROM` statement.

I then used the `WHERE` clause to filter employees belonging to the Marketing department. I also used the `AND` operator to add another condition, `office LIKE 'East%'`, to identify employees working in offices beginning with "East".

The `%` wildcard allows the query to match any additional characters following "East", including different office numbers.

This query allows me to identify employees working in the Marketing department who are located in East offices.

Retrieve employees in Finance or Sales

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Sales' OR department = 'Finance';
+-------------+--------------+----------+------------+-------------+
| employee_id | device_id    | username | department | office      |
+-------------+--------------+----------+------------+-------------+
|        1003 | d394e816f943 | sgilmore | Finance    | South-153   |
|        1007 | h174i497j413 | wjaffrey | Finance    | North-406   |
|        1008 | i858j583k571 | abernard | Finance    | South-170   |
|        1009 | NULL         | lrodriqu | Sales      | South-134   |
|        1010 | k2421212m542 | jlansky  | Finance    | South-109   |
|        1011 | l748m120n401 | drosas   | Sales      | South-292   |
|        1015 | p611q262r945 | jsoto    | Finance    | North-271   |
|        1017 | r550s824t230 | jclark   | Finance    | North-188   |
```

I used `SELECT *` to retrieve all columns from the `employees` table, which I specified using the `FROM` statement.

I then used the `WHERE` clause to filter employees belonging to the Sales department. I also used the `OR` operator to include employees belonging to the Finance department.

The `OR` operator allows the query to return records where either condition is true, meaning employees can belong to Sales or Finance to be included in the results.

This query allows me to identify employees working in either the Sales or Finance department.

Retrieve all employees not in IT

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE NOT department = 'Information Technology';
+-------------+-------------+----------+-----------------+-------------+
| employee_id | device_id   | username | department      | office      |
+-------------+-------------+----------+-----------------+-------------+
|        1000 | a320b137c219 | elarson  | Marketing       | East-170    |
|        1001 | b239c825d303 | bmoreno  | Marketing       | Central-276 |
|        1002 | c116d593e558 | tshah    | Human Resources | North-434   |
|        1003 | d394e816f943 | sgilmore | Finance         | South-153   |
|        1004 | e218f877g788 | eraab    | Human Resources | South-127   |
|        1005 | f551g340h864 | gesparza | Human Resources | South-366   |
|        1007 | h174i497j413 | wjaffrey | Finance         | North-406   |
|        1008 | i858j583k571 | abernard | Finance         | South-170   |
```

I used `SELECT *` to retrieve all columns from the `employees` table, which I specified using the `FROM` statement.

I then used the `WHERE` clause with the `NOT` operator to exclude employees belonging to the Information Technology department. The condition `NOT department = 'Information Technology'` ensures that only employees from other departments are included in the results.

This query allows me to identify all employees who work outside the Information Technology department.

Summary

Throughout this project, I used SQL statements such as `SELECT`, `FROM`, and `WHERE`, along with operators including `AND`, `OR`, `NOT`, and `LIKE`, to retrieve and filter information from the organisation's databases.

I investigated failed login attempts outside normal working hours, reviewed login activity on specific dates, and identified login attempts from outside Mexico. I also retrieved employee information based on specific departments and office locations.

This project helped me strengthen my SQL skills and develop a better understanding of how database queries can support cybersecurity investigations. It also gave me practical experience in retrieving relevant information efficiently to investigate potential security issues.